

Superscalar Execution, Out-of-Order Execution (OoO) & The Reorder Buffer

8.0 The IPC Barrier & The Limits of Scalar Pipelining

In Chapter 6, we mathematically proved the fundamental limit of the classical 5-stage pipeline: As the number of instructions approaches infinity, the Instructions Per Cycle (IPC) approaches an absolute asymptotic ceiling of 1.0. No matter how elegantly the code is written, a strict scalar pipeline can never retire more than one instruction per clock tick.

By the late 1980s and early 1990s, hardware architects realized that simply making the clock frequency faster (shrinking the clock period T_{clk}) was pushing silicon into severe thermal constraints. To double the performance of a CPU without touching the clock frequency, engineers had to break the IPC = 1.0 barrier. The solution was **Superscalar Architecture**.

Going Wider: The Superscalar Pipeline

A superscalar processor achieves an IPC greater than 1.0 by physically duplicating the execution hardware on the silicon die. Instead of fetching 1 instruction, decoding 1 instruction, and feeding it to 1 ALU, the entire pipeline is widened.

- **N-Wide Fetch & Decode:** The L1i cache bus is widened to fetch 4, 6, or 8 instructions simultaneously. Multiple parallel decoders translate these instructions simultaneously.
- **Parallel Execution Units:** The Execute stage is packed with multiple, specialized ports (e.g., 4 Integer ALUs, 2 Floating Point Units, 2 Address Generation Units).

If the CPU fetches four completely independent `add` instructions, it routes them to the four ALUs and executes them all in a single picosecond. The theoretical peak IPC jumps instantly from 1.0 to 4.0.

The VLIW Failure (Intel Itanium)

Early attempts to utilize multiple ALUs relied entirely on the software compiler. **VLIW (Very Long Instruction Word)** architectures, like the infamous Intel Itanium (IA-64), demanded that the C++ compiler bundle perfectly independent instructions into massive 128-bit bundles before the code was even executed. The assumption was that the compiler, having full visibility of the code, could schedule parallel execution perfectly.

The Itanium failed catastrophically. The compiler cannot predict *dynamic runtime behavior*. The compiler does not know if a memory pointer will trigger an L1 cache hit (1 ns) or an L3 cache miss (20 ns) until the program is physically running.

The Dynamic Dispatch Invariant: Static compilers (VLIW) failed because they attempt to schedule instructions at compile-time. Superscalar x86 architectures succeeded because they schedule instructions dynamically at run-time, reacting instantly to unpredictable physical cache latencies.

8.1 The In-Order Starvation Problem & Data-Flow

Moving away from VLIW, engineers built hardware that accepted standard sequential x86 assembly but attempted to dispatch it to multiple ALUs internally. The original Intel Pentium (1993) was an In-Order Superscalar processor. It had two execution pipelines (the U and V pipes). However, it executed code in strict programmatic order.

The Sequential Traffic Jam

Consider the following sequence fetched by a 4-wide in-order CPU:

```
Inst 1: load rax, [rcx]    // Cache Miss! Takes 200 cycles.
Inst 2: add rbx, rax       // RAW Hazard. Must wait for Inst 1.
Inst 3: sub r8, 5          // Mathematically Independent.
Inst 4: xor r9, r9         // Mathematically Independent.
```

In a strict In-Order Superscalar CPU, Instruction 1 stalls the entire pipeline while it waits for main memory. Because Instruction 2 depends on 1, it cannot execute. But what about Instructions 3 and 4? They are completely independent. They need nothing from `RAX`. However, because the CPU enforces *In-Order Dispatch*, Instructions 3 and 4 are physically blocked from entering the empty ALUs until Instructions 1 and 2 clear the bottleneck.

A 4-wide in-order CPU spends 80% of its massive execution ports sitting totally idle, starving for data while waiting on sequential cache misses.

The Out-of-Order (OoO) Paradigm Shift

To keep a massive 6-wide or 8-wide superscalar engine fed, the CPU cannot stop for cache misses. It must look ahead in the instruction stream, find instructions that are ready to execute (like Inst 3 and Inst 4), and fire them into the ALUs immediately, bypassing the stalled instructions entirely.

- **Control-Flow (Software):** "Execute Line 1, then Line 2, then Line 3."
- **Data-Flow (Hardware):** "Execute an instruction the absolute picosecond its required electrical voltages (operands) arrive, completely irrespective of where the instruction resides in the compiled binary."

By abandoning the programmer's sequence and adopting a pure Data-Flow execution model, the CPU maximizes ALU utilization. The challenge becomes how to safely execute code chaotically while maintaining the illusion of sequential logic to the OS.

The Wall of Memory: CPU frequencies scaled 1000x over 30 years, while DRAM latency scaled barely 10x. Out-of-Order execution was not invented just to run code faster; it was invented specifically to hide the catastrophic, ever-widening latency gap of main memory accesses.

8.2 The Three-Part OoO Engine

To execute chaos safely, modern architectures (like AMD Zen 4 or Apple M-Series) violently fracture the traditional 5-stage pipeline into three distinct phases:

1. **The In-Order Front-End:** Fetches and decodes instructions sequentially, predicting branches to keep a massive buffer of upcoming work filled.
2. **The Out-of-Order Execution Core:** A chaotic waiting room. Instructions sit here monitoring data buses. As soon as their data arrives, they fire out of sequence.
3. **The In-Order Back-End:** The architectural garbage collector. It takes the chaotically finished instructions, sorts them back into their original programmatic order, and commits them.

8.3 The Front-End: Decoupling Fetch & Decode

The Front-End's only job is to aggressively decode x86 assembly into micro-operations (μops) faster than the Execution Core can consume them. However, decoding variable-length CISC instructions is incredibly slow, complex, and power-hungry.

The Micro-Op Queue (μop Queue)

If the Fetch unit suffers an L1i cache miss, or the Branch Predictor stalls for a cycle to evaluate a jump target, the decoders stop producing μops . If the Front-End stops, the Execution Core starves instantly.

To prevent this, architects place a massive FIFO buffer (the μop Queue) between the Front-End and the Execution Core. The Front-End pumps decoded μops into the top of the queue. The Execution Core pulls μops from the bottom. This buffer mathematically decouples the two engines. The Front-End can stall for several cycles without the Execution Core ever noticing, as it simply continues eating from the buffer.

The L0 Micro-Op Cache (Decoded ICache)

Introduced effectively by Intel in the Sandy Bridge architecture, the L0 Micro-Op Cache bypasses the decoding penalty entirely. Decoding complex x86 instructions consumes up to 20% of the CPU's total thermal and power budget.

The L0 cache stores the post-decoded μops of recently executed instructions. If a C# `for` loop repeats, the CPU completely powers down the complex x86 Decoders. It fetches the pre-digested μops directly from the L0 cache, massively reducing heat, eliminating decode bottlenecks, and flooding the μop Queue with ready-to-execute logic.

The RISC Advantage: ARM processors (like Apple M3) have a massive advantage in the Front-End. Because ARM instructions are a fixed 32-bits, decoding is mechanically trivial. Apple can build 8-wide decoders easily. Intel struggles to build decoders wider than 4 or 6 because finding the instruction boundaries of variable-length x86 code requires massive, overlapping, power-hungry speculative decoding logic.

8.4 Allocation & The Physical Register File (PRF)

Once a μop leaves the queue, it enters the **Allocate/Rename Stage**. This is the exact boundary where In-Order execution ends and Out-of-Order chaos begins. Before an instruction can be scrambled, its artificial dependencies must be severed.

As discussed in Chapter 7, the x86 architecture is severely crippled by having only 16 general-purpose architectural registers (`RAX` , `RBX` , `RCX` , etc.). The software compiler is forced to aggressively reuse `RAX` over and over again, creating Write-After-Write (WAW) and Write-After-Read (WAR) hazards.

The Hardware Lie: The PRF

To break these artificial shackles, the CPU possesses a massive **Physical Register File (PRF)** containing 160 to 300+ hidden silicon registers (e.g., `P0` through `P256`). The operating system and your compiled application have absolutely no knowledge of these registers. They are dynamically managed purely in hardware.

8.5 The Register Alias Table (RAT) Mechanics

The RAT is a lightning-fast hardware mapping table. It intercepts every single μop entering the OoO core and rewrites its operands on the fly.

Assume the following sequential C# code compiles to:

```
Inst 1: mov eax, 5
Inst 2: add ebx, eax
Inst 3: mov eax, 10    // WAW Hazard! Compiler reused EAX.
```

Step 1: Allocation. The RAT processes Inst 1. It sees a write to `EAX`. It pops a clean, unused physical register off the hardware **Free List** (let's say `P42`). It records in its table: "EAX currently maps to P42". It alters the μop to `mov P42, 5`.

Step 2: Consumption. The RAT processes Inst 2. It sees a read from `EAX`. It looks up its table, sees `EAX` maps to `P42`, and alters the instruction to `add ebx, P42`.

Step 3: Re-Allocation. The RAT processes Inst 3. It sees another write to `EAX`. Instead of overwriting `P42` (which would destroy Inst 2's dependency), it pops a brand new register from the Free List (e.g., `P89`). It updates the table: "EAX now maps to P89". The instruction becomes `mov P89, 10`.

By mapping the second `EAX` write to an entirely different physical silicon location (`P89`), the WAW hazard instantly evaporates. Instruction 3 can now execute out-of-order, completely bypassing Instruction 2, without corrupting the original value in `P42`. The RAT has transformed sequential assembly into a pure, dependency-free Data-Flow graph.

Static Single Assignment (SSA): If you study compiler optimization (LLVM), you know compilers translate your code into SSA form, where every variable is assigned exactly once (e.g., `x1`, `x2`). The RAT is literally executing SSA transformation in real-time physical hardware at 5 billion times a second.

8.6 Reservation Stations & The Scheduler

After the RAT severs the false dependencies, the μop is dispatched into the **Scheduler**, physically landing in a **Reservation Station (RS)**. Reservation Stations were first introduced in 1967 in Robert Tomasulo's legendary algorithm for the IBM System/360 Model 91 floating-point unit.

The Holding Pen

Think of Reservation Stations as the ultimate waiting room. A μop sits in its slot, holding its physical register names.

- If its data is already computed (e.g., adding two constants, or reading a register that was populated cycles ago), the μop is immediately marked *Ready*.
- If it is waiting on data (e.g., waiting for P42 to be computed by an ALU, or waiting for a massive L3 cache miss to return), it sits completely dormant. It does not block the pipeline. It simply waits, continuously listening to the internal broadcast network.

8.7 The Common Data Bus (CDB) Wakeup & Select

The Common Data Bus (CDB) is the massive internal broadcast network of the Execution Core. It connects the output pins of every single ALU, FPU, and Cache directly to the inputs of all Reservation Stations simultaneously.

The Wakeup Phase

When an ALU finishes a calculation (e.g., it computes that P42 equals 0xFF), it broadcasts this payload across the entire die via the CDB: **[Tag: P42, Value: 0xFF]**.

Every single dormant μop sitting in a Reservation Station is physically wired to compare its required operand tags against the broadcast tag. If a μop is waiting for P42, the hardware comparator fires, and the μop instantly latches the broadcasted voltage 0xFF into its local buffer. The μop 's state flips to *Ready*. This is the **Wakeup**.

The Select Phase

Every single clock cycle (0.2 ns), the hardware Scheduler scans all the Reservation Stations. It looks for μops that have all their operands marked *Ready*. It selects the absolute oldest ready μops and forcefully injects them into the Execution Ports (ALUs).

Superscalar Burst Fire (Emerald)

If an L3 cache miss finally resolves and broadcasts a highly contended piece of data on the CDB, it might wake up 6 different μops simultaneously. On the very next clock cycle, the Select logic will pull all 6 disparate, out-of-order μops and fire them into 6 different ALUs concurrently, achieving an IPC of 6.0 for that specific nanosecond, perfectly compensating for the previous stalls.

Tomasulo's Algorithm: The combination of Register Renaming to break WAW/WAR dependencies, Reservation Stations to queue instructions dynamically, and the Common Data Bus to broadcast results is the holy trinity of Out-of-Order execution, collectively known as Tomasulo's Algorithm.

8.8 Memory Disambiguation: The Load/Store Queue (LSQ)

Executing ALUs out of order is mathematically safe thanks to the RAT. Registers have explicit names (EAX). But reading and writing to RAM out of order is an architectural nightmare. Memory addresses are dynamic; the CPU does not know the physical memory address of a pointer until the Address Generation Unit (AGU) actually executes the math to compute it.

The Pointer Aliasing Hazard

```
Inst 1: STORE 0xFF -> [RAX]
Inst 2: LOAD  [RBX] -> RCX
```

Can the Out-of-Order engine execute the LOAD (Inst 2) before the STORE (Inst 1)? If `RAX` and `RBX` point to completely different memory addresses, yes. This is safe and massively boosts throughput. But what if `RAX == RBX`? This is called **Pointer Aliasing**.

If the CPU executes the LOAD from `RBX` before the STORE to `RAX` completes, the LOAD grabs stale, corrupted data from the cache. The program's state is ruined.

The Load/Store Queue (LSQ)

To resolve this, memory operations bypass standard Reservation Stations and enter a highly specialized buffer called the **Load/Store Queue (LSQ)**. The LSQ enforces strict memory ordering models while permitting extreme speculation.

- When a LOAD $\mu\text{text{op}}$ computes its physical address in the AGU, it scans the Store portion of the LSQ. It asks: *"Are there any older STOREs waiting to execute whose memory addresses are currently unknown?"*
- **Memory Disambiguation:** Modern CPUs aggressively gamble. The hardware assumes `RAX != RBX` (that pointers rarely overlap). It allows the LOAD to execute speculatively, fetching the data from the L1d cache, even if older STOREs haven't resolved their addresses yet.

Memory Ordering Models (TSO vs. Weak)

The rules the LSQ must follow depend on the ISA. x86-64 uses **Total Store Ordering (TSO)**. Under TSO, LOADs can execute before older STOREs (as long as they don't alias), but STOREs can never execute before older STOREs. ARM architectures use a Weakly Ordered model, allowing the LSQ to aggressively reorder both LOADs and STOREs, placing the burden of memory fencing entirely on the software developer.

Memory Ordering Violations: What happens when the CPU guesses wrong? Eventually, the older STORE computes its address and realizes `RAX == RBX`. The LSQ hardware detects that a younger LOAD stole data from the cache prematurely. The Memory Order Buffer (MOB) triggers a catastrophic pipeline flush, destroying all speculative work and forcing the LOAD to re-execute.

8.9 Store-to-Load Forwarding

Let's look at the opposite scenario in the Load/Store Queue. What happens if the CPU knows for an absolute fact that the pointers *do* overlap?

```
Inst 1: STORE 0xFF -> [RAX]
Inst 2: LOAD  [RAX] -> RCX
```

Instruction 1 wants to write to an address. Instruction 2 wants to read from that exact same address immediately after. In a naive pipeline, Instruction 2 must wait for Instruction 1 to send the data all the way to the L1 Data Cache, and then Instruction 2 must issue a cache read to pull it back out. This round-trip to the L1 Cache takes 4 to 5 clock cycles.

To avoid this massive latency penalty, the LSQ utilizes **Store-to-Load Forwarding**. When the LOAD (Inst 2) calculates its address in the AGU, it snoops the Store Queue. It finds Inst 1 sitting in the queue, waiting to be written to memory. Because the addresses perfectly match, the LSQ physically intercepts the `0xFF` data payload sitting inside the pending Store Buffer and wires it *directly* into the LOAD's destination physical register.

The LOAD instruction executes instantly in 0 cycles, completely bypassing the L1 Cache hierarchy. The STORE still eventually commits to the L1 cache in the background, but the pipeline throughput is completely unblocked.

The Forwarding Failure (Orange)

Store-to-Load forwarding only works if the LOAD perfectly aligns with the STORE. If you store a 64-bit integer, but then attempt to LOAD the upper 32-bits of that address, the hardware forwarding logic usually cannot handle the byte-offset translation dynamically. The forward fails, and the CPU suffers a massive stall, forcing the LOAD to wait for the STORE to fully flush to the L1 cache before reading it.



8.10 The Back-End: The Reorder Buffer (ROB) & Retirement

If instructions execute chaotically out of order, how does a debugger step through code line by line? How does the operating system survive a Page Fault if the memory access occurred on an instruction that executed 10 cycles ahead of schedule?

The answer is the **Reorder Buffer (ROB)**. The ROB is the ultimate guarantor of architectural sanity. It enforces the golden rule of modern processors: *Chaos in execution, perfect order in retirement*.

The Circular Queue

The ROB is a massive hardware circular queue (often 200 to 300 entries deep). When a μop is decoded in the In-Order Front-End, it is sequentially allocated a slot at the tail of the ROB. It receives a physical ROB Ticket.

When the μop fires out-of-order in the Execution Core, it computes its result. However, it **does not** write this result to the architectural state (the registers the OS can actually see, like the true `RAX`). It writes the result only to its hidden Physical Register (PRF) and marks its ROB Ticket as *Completed*.

The Architectural Commit (Retirement)

The In-Order Back-End of the CPU monitors the *Head* of the ROB. The CPU only allows an instruction to **Retire** (officially map its Physical Register back to the Architectural Register state) if:

1. It is located exactly at the Head of the ROB.
2. It has successfully completed execution without exceptions.
3. All instructions sequentially older than it have already retired.

If a fast XOR instruction finishes in 1 cycle, but is sitting behind a slow LOAD instruction waiting on a 200-cycle RAM fetch, the fast instruction sits completely trapped in the ROB. It cannot retire until the slow instruction finishes and leaves the queue. The architectural state updates purely, strictly sequentially.

ROB Exhaustion: If a cache miss takes 400 cycles, the OoO core keeps executing younger instructions and stuffing their completed results into the ROB. Eventually, the 200-slot ROB fills completely up. The Allocation stage hits a wall, and the entire CPU hard-stalls. This is why scaling cache misses is devastating; it physically overflows the CPU's ability to buffer chaos.

8.11 Precise Exceptions & Speculation Recovery

The true genius of the Reorder Buffer is not just ordering; it is the implementation of **Precise Exceptions** and the flawless recovery from branch mispredictions.

Handling Hardware Faults

Imagine an Out-of-Order execution core evaluating a `div eax, ebx` instruction. The ALU executes the math, but `EBX` is zero. The ALU triggers a Hardware Divide-by-Zero Exception.

Because the CPU is out-of-order, there might be 50 younger instructions that have already executed speculatively *after* the divide instruction. If the CPU panicked instantly, those 50 instructions would corrupt the memory state before the OS could handle the crash.

Instead, the Execute unit simply flags the Divide instruction's ROB entry as "Poisoned with Exception". The instruction sits quietly in the queue. Only when the poisoned instruction reaches the absolute *Head* of the ROB does the CPU trigger the actual interrupt to the OS. Because it is at the head, the architectural state is mathematically perfectly pristine—exactly as it was immediately prior to the crash. The OS fault handler takes over safely.

Branch Misprediction Recovery

The exact same physical mechanic handles Branch Prediction.

If the Front-End guesses a branch wrong, it fetches and executes 100 instructions down the wrong path. They finish out-of-order and sit in the ROB. When the actual branch instruction finally evaluates in the ALU and realizes the misprediction, it flags itself in the ROB.

When the branch reaches the Head of the ROB, the CPU clears the entire ROB queue behind it. It resets the RAT (abandoning all the newly mapped Physical Registers, dumping them back onto the Free List). The 100 speculative instructions are vaporized before they can ever Retire. The architectural state remains perfectly isolated from the speculative chaos.

8.12 The Vulnerability of Speculation: Spectre

In 2018, the computer science world was shaken by the disclosure of the **Spectre** and **Meltdown** hardware vulnerabilities. They were not software bugs; they were fundamental physics exploits leveraging the Reorder Buffer (ROB) and Out-of-Order execution.

The Microarchitectural Footprint

We established that if the CPU speculatively executes down the wrong branch path, the ROB perfectly vaporizes the *Architectural State* (the Registers) when the misprediction is caught. The software execution history is pristine.

However, the ROB only rolls back registers. It does **not** roll back *Microarchitectural State*. If a speculative instruction executes a memory LOAD, the physical hardware pulls that data from Main RAM and places it into the **L1 Data Cache**. When the ROB flushes, it clears the register, but the data remains physically sitting inside the L1 Cache.

The Flush+Reload Attack Vector

A hacker writes a malicious script with an `if (x < array_size)` bounds check. They train the CPU's branch predictor to assume the check is always true. Then, they request an out-of-bounds read (e.g., stealing a kernel password at `array_size + offset`).

The branch predictor guesses true, speculatively executing the malicious read out-of-order. The CPU fetches the password into the L1 Cache. It then uses the value of the password to load a secondary array: `dummy_array[password_byte]`.

A few cycles later, the ALU resolves the `if` condition, realizes `x` was out-of-bounds, and triggers a ROB flush. The registers are vaporized. But the cache lines for `dummy_array[password_byte]` remain loaded.

The hacker then iterates through every index of `dummy_array`, measuring the exact nanosecond timing of the memory accesses. The index that returns instantly (because it is in the L1 cache) is the value of the password byte. By abusing the OoO engine's quest for extreme IPC, attackers shattered the fundamental security isolation of modern processors.

Hardware Mitigations: Fixing Spectre required massive microcode updates that forced the CPU to insert artificial serialization barriers (lfence/mfence), aggressively preventing the OoO engine from speculating on sensitive memory loads. This caused severe performance regressions (10-20%) across cloud datacenters globally.

8.13 Software Relevancy: Maximizing IPC in Code

The Out-of-Order core is a miracle of engineering, but it has finite limits. The ROB can only hold ~200 instructions. The Reservation Stations can only hold ~100. If your code features a tight dependency chain of 30 sequential math operations, the OoO core cannot parallelize it. The Reservation Stations will fill up with dormant instructions waiting on the CDB, the allocator will stall, and the massive 6-wide superscalar engine will drop from a theoretical IPC of 6.0 down to an abysmal IPC of 0.5.

Pointer Aliasing & The `restrict` Keyword (C/C++)

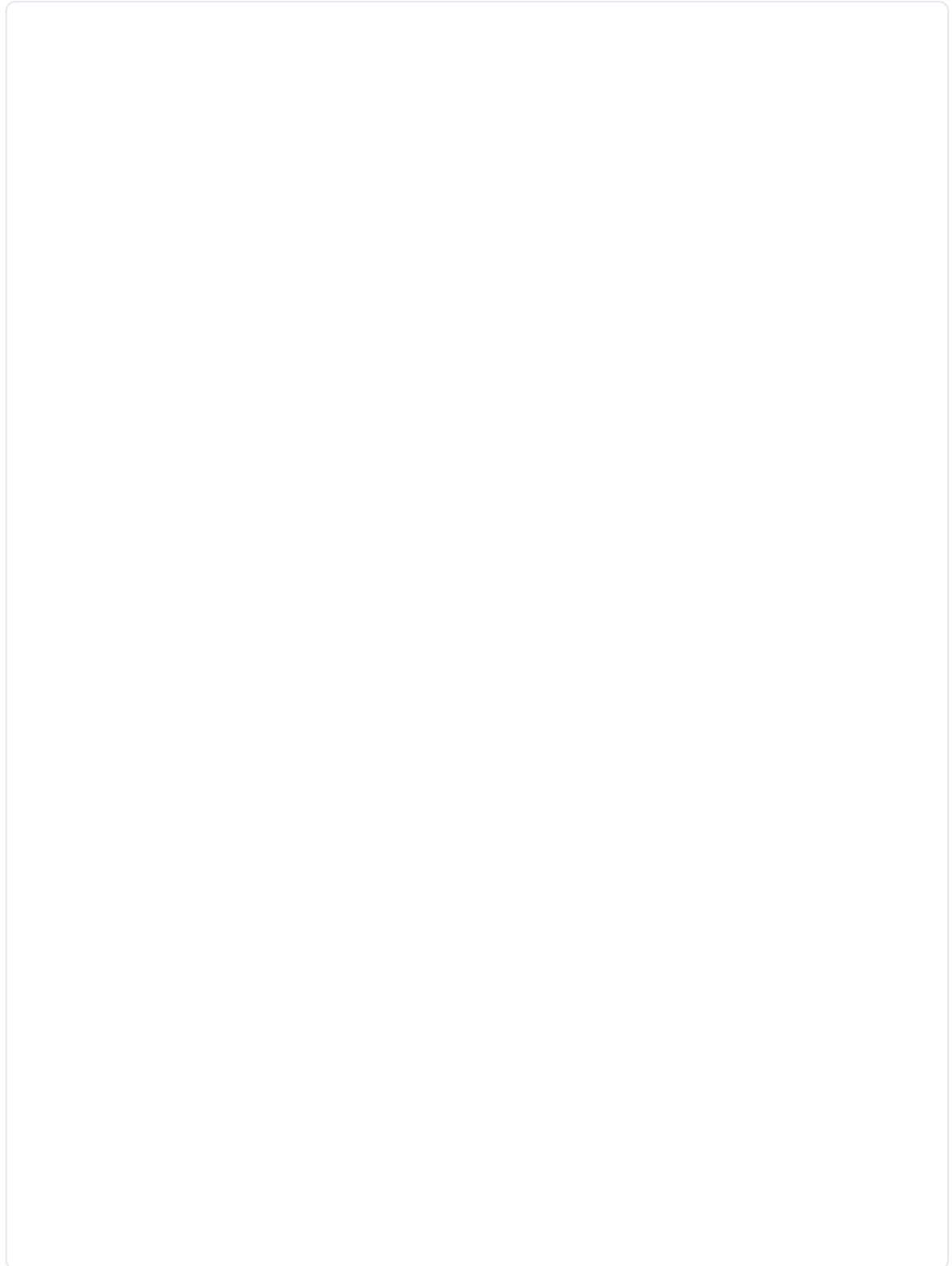
We discussed how the Load/Store Queue (LSQ) must guess if pointers alias (overlap). In languages like C and C++, the compiler must be deeply pessimistic. If you pass two array pointers into a function, the compiler assumes they might overlap, forcing the CPU to serialize the memory accesses to prevent Memory Ordering Violations.

```
void AddArrays(int* restrict a, int* restrict b, int* restrict result, int len) {
    for (int i = 0; i < len; i++) {
        result[i] = a[i] + b[i];
    }
}
```

By adding the `restrict` keyword, the software engineer provides a legally binding contract to the compiler that these pointers will never alias. The compiler aggressively reorders the assembly, and the LSQ violently parallelizes the memory fetches, often resulting in a 3x to 4x throughput increase via vectorization.

The Rust Advantage (Emerald)

This aliasing problem is exactly why idiomatic Rust frequently achieves tighter optimization and higher IPC than C++. Rust's strict borrowing and ownership rules mathematically guarantee to the LLVM compiler that there is no mutable aliasing at compile time. The compiler automatically emits `noalias` flags, unleashing the full parallel fury of the CPU's Out-of-Order execution core without manual annotation.



8.14 Chapter Verification, Checklist & Quiz

Verification Checklist

I can explain why the $\text{IPC} = 1.0$ limit of a scalar pipeline forced the invention of Superscalar processors.

I understand why VLIW architectures (Itanium) failed compared to dynamic Superscalar x86 architectures.

I can explain the shift from Control-Flow (sequential code) to Data-Flow (execute when ready) in OoO cores.

I understand how the μop Queue decouples the In-Order Front-End from the OoO Core.

I can explain how the RAT maps Architectural Registers to the Physical Register File (PRF).

I can describe the Wakeup & Select logic of Reservation Stations monitoring the Common Data Bus (CDB).

I understand the Pointer Aliasing hazard and why Memory Disambiguation is required in the LSQ.

I can explain Store-to-Load Forwarding within the Load/Store Queue.

I understand the fundamental rule of the Reorder Buffer (ROB): Chaos in Execution, Order in Retirement.

I understand how the Spectre vulnerability exploits the microarchitectural L1 Cache footprint left behind by speculative execution flushes.

Comprehensive Examination

1. What was the fatal flaw of early "In-Order" Superscalar processors (like the original Pentium) that prevented them from fully utilizing their parallel execution units?

- A) They lacked sufficient L1 Cache size to feed multiple ALUs.
- B) They required programmers to manually thread assembly code across execution units.
- C) A single data dependency or cache miss on the leading instruction would rigidly block all subsequent, mathematically independent instructions from entering the ALUs, starving the parallel ports.
- D) The Register Alias Table (RAT) was too slow to map 16 architectural registers.

2. What was the primary reason Intel's Itanium (IA-64) VLIW architecture failed in the market compared to AMD's dynamic Superscalar architecture?

- A) VLIW CPUs consumed too much power.
- B) VLIW relied strictly on the software compiler to schedule parallel instruction bundles at compile-time. Compilers cannot predict dynamic runtime cache misses, causing the bundles to shatter and stall the CPU.
- C) VLIW instructions could not be translated into micro-operations (μops).
- D) The OS kernels refused to support 128-bit instructions.

3. In the Out-of-Order core, what specific event triggers an instruction sitting in a Reservation Station to transition to the "Ready" state (The Wakeup phase)?

- A) The Reorder Buffer (ROB) signals that the instruction has reached the head of the queue.
- B) The Instruction Fetch unit retrieves the next cache line.
- C) Another ALU completes a calculation and broadcasts the required physical register ID and its voltage onto the Common Data Bus (CDB).
- D) The LSQ confirms that there are no pointer aliasing hazards.

4. Why must memory operations (LOADs and STOREs) bypass standard Reservation Stations and enter a dedicated Load/Store Queue (LSQ)?

- A) Memory operations require 64-bit ALUs, whereas Reservation Stations only support 32-bit operations.
- B) The CPU cannot statically determine if two memory pointers point to the exact same physical RAM address until execution. This creates Pointer Aliasing hazards that require specialized disambiguation logic to prevent data corruption.
- C) The LSQ encrypts memory payloads before writing them to the L1 Cache.
- D) Memory operations do not use Physical Registers (PRF).



5. An instruction triggers a Divide-by-Zero exception in the Execute stage of an Out-of-Order CPU. However, 15 younger instructions have already speculatively executed out-of-sequence. How does the CPU recover perfectly?
- A) It throws a hardware interrupt and immediately kills the OS thread.
 - B) The CPU flags the instruction as "poisoned" inside the Reorder Buffer (ROB). When the poisoned instruction reaches the head of the ROB to Retire, the CPU throws the exception and flushes the entire ROB, ensuring none of the 15 speculative instructions ever update the architectural state.
 - C) The CPU relies on the L1 cache to revert the physical memory state.
 - D) It uses the Common Data Bus to broadcast an undo command to the ALUs.
6. How did the Spectre vulnerability fundamentally exploit the physical mechanics of the Reorder Buffer (ROB) and Out-of-Order speculation?
- A) By overflowing the ROB's 200-entry capacity to trigger an integer overflow.
 - B) By altering the Free List of the RAT to map user registers to kernel memory addresses.
 - C) By recognizing that while the ROB perfectly rolls back architectural registers during a flush, it leaves microarchitectural footprints (data pulled into the L1 Cache) fully intact, which can be extracted via timing side-channels.
 - D) By corrupting the Wakeup & Select logic in the Reservation Stations.

Systems Writing Prompts

Prompt 1: Tracking the OoO Lifecycle

Trace the exact lifecycle of an instruction through an Out-of-Order CPU. Specifically highlight its interactions with the μop Queue, the RAT (Physical Register Allocation), the Reservation Stations (CDB Wakeup), and its ultimate retirement at the Reorder Buffer.

Prompt 2: LSQ Store-to-Load Forwarding

Explain the physics of Store-to-Load Forwarding. If Instruction 1 is `STORE [Address X]` and Instruction 2 is `LOAD [Address X]`, how does the Load/Store Queue intercept this dependency to save latency without waiting for the L1d cache to actually ingest the Store data?

CHAPTER 8 TECHNICAL ANSWER KEY

1. (C) In-order execution forces rigid dependency resolution. If 4 execution ports are ready, but the leading instruction misses the cache, the entire 4-wide block stalls. Out-of-Order execution was mandatory to bypass these stalled instructions and feed the starving execution ports with mathematically independent work located further down the code stream.
 2. (B) VLIW relied on software compilers to perfectly schedule parallel execution. However, compilers cannot predict dynamic memory behaviors (cache misses, page faults). When the compiled bundle encountered a runtime delay, the entire rigid bundle stalled. Superscalar x86 delegates this scheduling to the hardware (the Scheduler/ROB), which adapts to cache misses in real-time by firing independent instructions instead.
 3. (C) Reservation Stations act as a waiting room monitoring the Common Data Bus (CDB). When an ALU finishes computing a physical register (e.g., P42), it broadcasts the ID and voltage across the die. Any dormant instruction waiting for P42 "wakes up", latches the data, and alerts the Scheduler that it is ready to fire.
 4. (B) Registers have static names (EAX). Memory addresses are dynamic variables calculated at runtime by the Address Generation Unit. The CPU cannot definitively know if a STORE to [RAX] and a LOAD from [RBX] collide until the math executes. The LSQ enforces order, disambiguates pointers, and handles the catastrophic MOB flush if a speculative LOAD illegally read from RAM before an older overlapping STORE wrote to it.
 5. (B) The ROB acts as a strict sequential filter protecting the architectural state. Speculative instructions write to physical registers, not architectural ones. When the poisoned Divide-by-Zero instruction reaches the head of the ROB, the OS fault is triggered, and all subsequent physical registers in the ROB are instantly dumped back into the Free List.
 6. (C) Speculative execution alters the physical state of the L1 Cache. Spectre leverages this by mis-training the branch predictor to execute an illegal, out-of-bounds array read. The ROB catches the error and cleanly flushes the registers, but the illegally accessed data now resides in the hyper-fast L1 cache. The attacker extracts it using cache-timing attacks (Flush+Reload), deducing the secret by observing how fast subsequent memory reads respond.
- Prompt 1:** The Front-End decodes x86 into $\mu\text{text}\{\text{ops}\}$ and banks them in the $\mu\text{text}\{\text{op}\}$ Queue. The RAT allocates a fresh physical register (PRF) and maps the architectural name to it. The $\mu\text{text}\{\text{op}\}$ is dispatched to a Reservation Station while simultaneously claiming a slot at the tail of the ROB. It waits on the CDB for data. Once awake, the Scheduler fires it into an ALU. It computes, broadcasts its result on the CDB to wake others, and marks its ROB ticket as completed. Finally, when it becomes the absolute oldest instruction at the Head of the ROB, it "Retires", permanently committing its physical register to the visible architectural state.
- Prompt 2:** When the LSQ AGU calculates the physical pointer for the LOAD, it scans the pending STORE queue. It discovers an older STORE targeting the exact same physical address that hasn't officially retired to the L1 cache yet. To prevent a memory ordering violation, the LSQ intercepts the payload sitting inside the pending STORE buffer and physically wires it directly into the LOAD's destination register. The LOAD executes instantly without ever touching the actual L1 Cache hierarchy, saving massive latency.